



TRABAJO PRÁCTICO INTEGRADOR

**Sistema de Semáforos Inteligentes y Análisis
Comparativo de Paradigmas**

**PARADIGMAS Y LENGUAJES
2026**

GRUPO N° 33

- Escobar, Mateo
- Ojeda, Agustina
- Romero, Marcos
- Silvia, Fernando
- Wozniuk, Abril

EL PROBLEMA DE LOS 3 FOCOS

El desarrollo del Sistema de Semáforos Inteligentes se articuló bajo una arquitectura estrictamente funcional, evitando la mutabilidad global mediante el uso de estructuras locales (let) y el paso de estados como argumentos. La lógica de transición y control, ejemplificada en funciones (valido-datos y transición), opera bajo principios los cuales tratan de una expresión puede ser reemplazada por su valor resultante sin alterar el comportamiento del programa, donde la validez de los estados es verificada mediante predicados que garantizan un comportamiento determinista, evitando cualquier alteración destructiva de las estructuras de datos. La gestión del tiempo y la planificación de los ciclos no dependen de variables mutables, sino de funciones puras y composiciones aritméticas (timer, duracion-ciclo), las cuales aprovechan la potencia de funciones de orden superior como mapcar para la auditoría y distribución temporal. Este enfoque garantiza que el sistema sea auditable y modular, facilitando tanto la validación de entrada de datos (siesentero, ingreso-ciclo) como el procesamiento continuo del flujo de tráfico sin comprometer la integridad de la memoria.

BITACORA DE DEPURACIÓN

REQUERIMIENTO 1:

En el desarrollo del requerimiento 1 (Estados de transición) tuvimos un inconveniente con la interpretación de la consigna, específicamente en el formato de retorno de la función.

En este caso, la salida se corresponde con una lista, si bien la información es correcta, el formato no es el esperado.

(Primera versión del requerimiento 1)

```
;; FUNCION: valido-datos
;; NATURALEZA: Pura (verifica si los datos pasados por parámetros son válidos)
;; ESTRATEGIA: Orden superior (combina una funcion condicional simple con el llamado a otra funcion si los datos son validos y un mensaje si los datos no son validos)
;; IMPACTO: No destructiva

(defun valido-datos (color-actual color-siguiente)

  (if (and (or (equalp color-actual 'en-rojo) (equalp color-actual 'en-amarillo)
              (equalp color-actual 'en-verde)) (or (equalp color-siguiente 'rojo)
              (equalp color-siguiente 'amarillo) (equalp color-siguiente 'verde))
      (not (equalp color-actual color-siguiente)) (transicion color-actual color-siguiente) "Error en el ingreso de datos"))

;; FUNCION: transición
;; NATURALEZA: Pura (muestra el color actual del semaforo y a que color cambia)
;; ESTRATEGIA: Orden superior (muestra el color actual del semaforo y a que color cambia)
;; IMPACTO: No destructiva

(defun transición (color-actual color-siguiente)
  (list color-actual "cambiar-a" color-siguiente))
```

En la versión, se incorporó una estructura COND que define explícitamente las transiciones permitidas, logrando un comportamiento más acorde al funcionamiento real de un semáforo. Además, la lógica de validación se volvió más clara y específica, permitiendo un mayor control sobre los datos de entrada y evitando cambios no contemplados.

(Segunda versión del requerimiento 1)

```
;; FUNCION: valido-datos
;; NATURALIEZA: Pura (verifica si los datos pasados por parametros son validos)
;; ESTRATEGIA: Orden superior (combina una funcion condicional simple con el llamado a otra funcion si los datos son validos y un mensaje si los datos no son validos)
;; IMPACTO: No destructiva

(defun valido-datos (color-actual color-siguiente)
  (if (and (or (equalp color-actual 'en-rojo)
               (equalp color-actual 'en-amarillo)
               (equalp color-actual 'en-verde))
          (or (equalp color-siguiente 'rojo)
              (equalp color-siguiente 'amarillo)
              (equalp color-siguiente 'verde))
      (and (not (and (equalp color-actual 'en-rojo) (equalp color-siguiente 'rojo))
                (not (and (equalp color-actual 'en-amarillo) (equalp color-siguiente 'amarillo))
                    (not (and (equalp color-actual 'en-verde) (equalp color-siguiente 'verde))))))
      (transicion color-actual color-siguiente) "Error en el ingreso de datos"))))

;; FUNCION: transicion
;; NATURALIEZA: Pura (muestra el color actual del semaforo y a que color cambia)
;; ESTRATEGIA: Orden superior (muestra el color actual del semaforo y a que color cambia)
;; IMPACTO: No destructiva

(defun transicion (color-actual color-siguiente)
  (cond
    ((and (equalp color-actual 'en-rojo) (equalp color-siguiente 'amarillo)) (format t "~a pasar-a-a" color-actual color-siguiente))
    ((and (equalp color-actual 'en-amarillo) (equalp color-siguiente 'verde)) (format t "~a pasar-a-a" color-actual color-siguiente))
    ((and (equalp color-actual 'en-verde) (equalp color-siguiente 'amarillo)) (format t "~a pasar-a-a" color-actual color-siguiente))
    ((and (equalp color-actual 'en-amarillo) (equalp color-siguiente 'rojo)) (format t "~a pasar-a-a" color-actual color-siguiente))
  )
)
```

Este requerimiento 1 se volvió a modificar para realizar mejoras orientadas a hacer el código más claro, mantenible y representativo del comportamiento del semáforo. En la función valido-datos se reorganizó la estructura lógica, agrupando las validaciones y utilizando un not sobre una expresión or para evitar que el color actual y el siguiente sean equivalentes, lo que facilita la lectura y comprensión del algoritmo. Por otra parte, la función transición dejó de mostrar mensajes mediante format y pasó a devolver listas con el estado de la transición, convirtiéndola en una función más funcional y reutilizable. Estas modificaciones hicieron que el código fuera más robusto, legible y fácil de mantener o ampliar en futuras versiones.

(Verión final del requerimiento 1)

```
[1]> ;; FUNCION: valido-datos
;; NATURALIEZA: Pura (verifica si los datos pasados por par metros son v lidos)
;; ESTRATEGIA: Orden superior (combina una funcion condicional simple con el llamado a otra funcion si los datos son validos y un mensaje si los datos no son validos)
;; IMPACTO: No destructiva

(defun valido-datos (color-actual color-siguiente)

  (if (and
      (or (equalp color-actual 'en-rojo)
          (equalp color-actual 'en-amarillo)
          (equalp color-actual 'en-verde))

      (or (equalp color-siguiente 'rojo)
          (equalp color-siguiente 'amarillo)
          (equalp color-siguiente 'verde))

      (not (or
            (and (equalp color-actual 'en-rojo)
                  (equalp color-siguiente 'rojo))

            (and (equalp color-actual 'en-amarillo)
                  (equalp color-siguiente 'amarillo))

            (and (equalp color-actual 'en-verde)
                  (equalp color-siguiente 'verde))))
      )

      (transicion color-actual color-siguiente)

      "Error en el ingreso de datos"))

VALIDO-DATOS
[2]>
;; FUNCION: transición
;; NATURALIEZA: Pura (muestra el color actual del semaforo y a que color cambia)
;; ESTRATEGIA: Orden superior (muestra el color actual del semaforo y a que color cambia)
;; IMPACTO: No destructiva

(defun transicion (color-actual color-siguiente)

  (cond

    ((and (equalp color-actual 'en-rojo)
          (equalp color-siguiente 'amarillo))
     (list color-actual "cambiar-a-amarillo"))

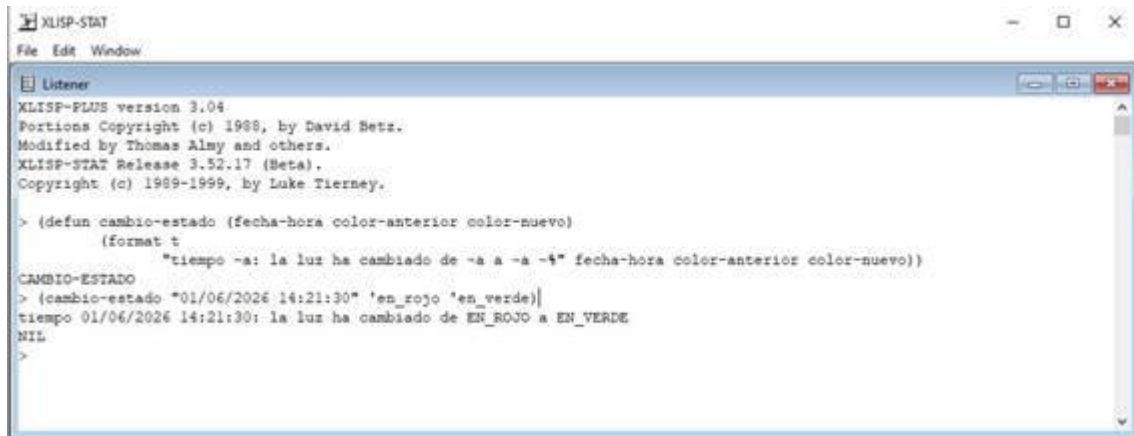
    ((and (equalp color-actual 'en-amarillo)
          (equalp color-siguiente 'verde))
     (list color-actual "cambiar-a-verde"))

    ((and (equalp color-actual 'en-verde)
          (equalp color-siguiente 'rojo))
     (list color-actual "cambiar-a-rojo"))

    (t
     (list color-actual 'accion-por-defecto))))
```

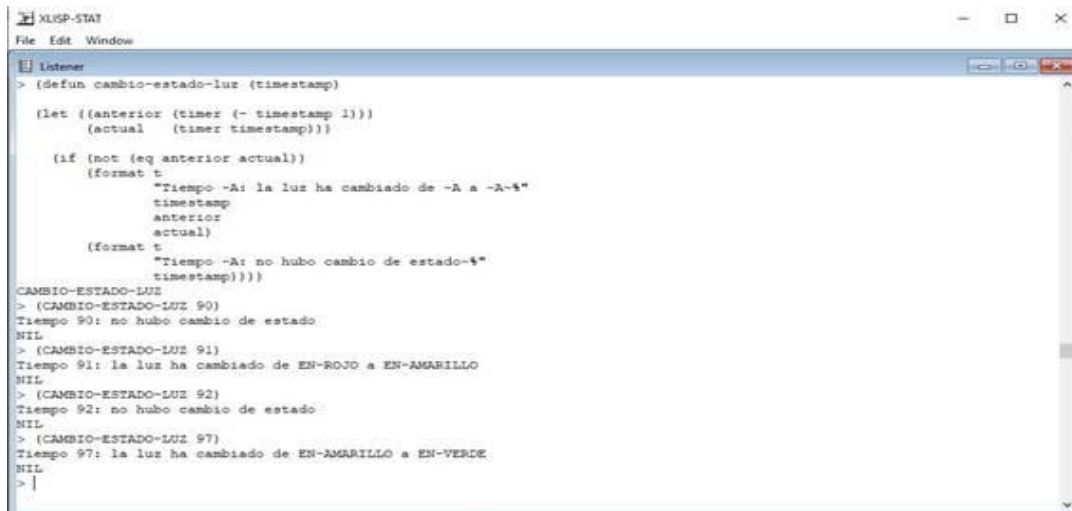
REQUERIMIENTO 3 - sin correcciones

```
;; =====  
;; FUNCIÓN: cambio-estado  
;; NATURALEZA: Impura (realiza salida por pantalla mediante format)  
;; ESTRATEGIA: Directa (muestra un mensaje con los datos recibidos)  
;; IMPACTO: No destructiva  
;; =====
```



```
XLISP-STAT  
File Edit Window  
Listener  
XLISP-PLUS version 3.04  
Portions Copyright (c) 1988, by David Betz.  
Modified by Thomas Almy and others.  
XLISP-STAT Release 3.52.17 (Beta).  
Copyright (c) 1988-1999, by Luke Tierney.  
  
> (defun cambio-estado (fecha-hora color-anterior color-nuevo)  
  (format t  
    "tiempo -a: la luz ha cambiado de -a a -a -t" fecha-hora color-anterior color-nuevo))  
CAMBIO-ESTADO  
> (cambio-estado "01/06/2026 14:21:30" 'en_rojo 'en_verde)  
tiempo 01/06/2026 14:21:30: la luz ha cambiado de EN_ROJO a EN_VERDE  
NIL  
>
```

Esta función simplemente imprime un mensaje, que se le pasa manualmente como la fecha u hora, el color anterior y el color nuevo, como el ejemplo que se ve en la imagen. Las limitaciones que presenta son que, no verifica si realmente hubo un cambio, no calcula los colores, depende completamente de los datos que recibe.

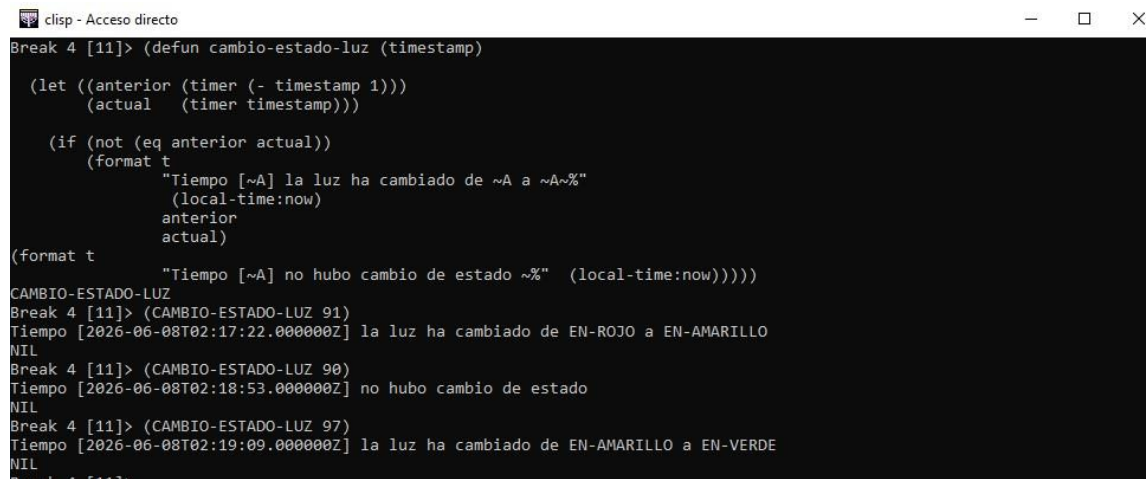


```
XLISP-STAT  
File Edit Window  
Listener  
> (defun cambio-estado-luz (timestamp)  
  (let ((anterior (timer (- timestamp 1)))  
        (actual (timer timestamp)))  
    (if (not (eq anterior actual))  
        (format t  
          "Tiempo -A: la luz ha cambiado de -A a -A-t"  
            timestamp  
            anterior  
            actual)  
        (format t  
          "Tiempo -A: no hubo cambio de estado-t"  
            timestamp))))  
CAMBIO-ESTADO-LUZ  
> (CAMBIO-ESTADO-LUZ 90)  
Tiempo 90: no hubo cambio de estado  
NIL  
> (CAMBIO-ESTADO-LUZ 91)  
Tiempo 91: la luz ha cambiado de EN-ROJO a EN-AMARILLO  
NIL  
> (CAMBIO-ESTADO-LUZ 92)  
Tiempo 92: no hubo cambio de estado  
NIL  
> (CAMBIO-ESTADO-LUZ 97)  
Tiempo 97: la luz ha cambiado de EN-AMARILLO a EN-VERDE  
NIL  
> |
```

El cambio que se realizó a la función del requerimiento 3 se hizo para que este obtenga automáticamente, necesitando recibir como parámetro timestamp, el color en el instante anterior, el color en el instante actual y luego los compara, si detecta que son diferentes, muestra un mensaje indicando el momento en que ocurrió el cambio y los estados involucrados. En caso de que ambos sean iguales, informa que en ese instante no hubo modificación en el estado de la luz. De esta manera, la función permite monitorear la evolución del semáforo a lo largo del tiempo sin alterar los datos utilizados.

AUTONOMÍA Y ECOSISTEMA

Para cumplir con los requisitos de la Fase 2, el grupo ha seleccionado la librería local-time sobre cl-json. Esta decisión se fundamenta en la necesidad de optimizar el Sistema de Auditoría, por lo cual se investigó e instaló Quicklisp como gestor de paquetes para Common Lisp. Una vez instalado, se incorporó la librería Local-Time al proyecto mediante el comando (ql:quickload "local-time"). La integración de esta librería se aplicó a la función cambio-estado-luz, encargada de auditar los cambios de estado del semáforo. Originalmente, la función únicamente mostraba el valor numérico del timestamp recibido, lo que dificultaba la interpretación temporal de los eventos. Luego de incorporar Local-Time (local-time:now), se modificó la salida de la función para que, además de informar el cambio de color, muestre la fecha y hora en un formato legible para el usuario. El proceso consistió en instalar Quicklisp, cargar la librería Local-Time, verificar su correcto funcionamiento y finalmente utilizar sus funciones de obtención y formateo de fecha y hora dentro de la auditoría. Como resultado, el sistema genera registros más claros y comprensibles, facilitando el seguimiento de los cambios de estado del semáforo durante la ejecución de la simulación.



```
clisp - Acceso directo
Break 4 [11]> (defun cambio-estado-luz (timestamp)
  (let ((anterior (timer (- timestamp 1)))
        (actual (timer timestamp)))
    (if (not (eq anterior actual))
        (format t
          "Tiempo [~A] la luz ha cambiado de ~A a ~A~%"
          (local-time:now)
          anterior
          actual)
        (format t
          "Tiempo [~A] no hubo cambio de estado ~%" (local-time:now))))))
CAMBIO-ESTADO-LUZ
Break 4 [11]> (CAMBIO-ESTADO-LUZ 91)
Tiempo [2026-06-08T02:17:22.000000Z] la luz ha cambiado de EN-ROJO a EN-AMARILLO
NIL
Break 4 [11]> (CAMBIO-ESTADO-LUZ 90)
Tiempo [2026-06-08T02:18:53.000000Z] no hubo cambio de estado
NIL
Break 4 [11]> (CAMBIO-ESTADO-LUZ 97)
Tiempo [2026-06-08T02:19:09.000000Z] la luz ha cambiado de EN-AMARILLO a EN-VERDE
NIL
Break 4 [11]>
```

ESTUDIO COMPARATIVO

En esta sección, se realizará una breve presentación del lenguaje Erlang y una comparación con Lisp, dos lenguajes que pertenecen al paradigma de programación funcional, aunque con enfoques y características particulares. Se analizarán algunos de los aspectos más relevantes de Erlang, como su sintaxis, su semántica y los principios sobre los que se basa su funcionamiento, haciendo especial énfasis en las decisiones de diseño que influyeron en la resolución del problema planteado. Asimismo, se establecerán comparaciones con Lisp para destacar las diferencias en la forma de escribir y estructurar el código, así como el impacto que estas características tuvieron durante el desarrollo de la solución propuesta.

ERLANG - PRESENTACIÓN

ERLANG es un lenguaje de programación diseñado por Ericsson a finales de los años 80. Fue creado con un propósito muy claro: construir sistemas masivos concurrentes, tolerantes a fallos y que nunca deje de funcionar. ERLANG nació para resolver los problemas de las redes de comunicación.

Es un lenguaje funcional, declarativo, orientado a la concurrencia, de asignación única y con un modelo de ejecución basado en Actores (procesos ligeros) que se comunican mediante paso de mensajes en: lugar de compartir memoria.

No está diseñado para crear interfaces gráficas vistosas ni para desarrollo web ligero; su fuerte es la infraestructura: servidores de chat (como los cimientos de WhatsApp), sistemas bancarios, videojuegos multijugador masivos y redes de telefonía.

CARACTERÍSTICAS PRINCIPALES

Erlang se apoya en tres pilares fundamentales: la concurrencia, la tolerancia a fallos y la distribución, características que lo convierten en una opción adecuada para el desarrollo de sistemas de alta disponibilidad. A diferencia de otros lenguajes que utilizan los hilos del sistema operativo, Erlang implementa procesos ligeros gestionados por su máquina virtual BEAM, permitiendo ejecutar millones de procesos concurrentes con un bajo consumo de memoria. Su filosofía de tolerancia a fallos, conocida como "*Let it crash*", propone que los procesos que presentan errores finalicen su ejecución y sean reiniciados automáticamente por procesos supervisores, garantizando la continuidad del sistema. Además, los procesos están completamente aislados y no comparten memoria, comunicándose únicamente mediante el envío de mensajes, lo que evita que un fallo se propague al resto de la aplicación. Entre sus características más destacadas también se encuentran la actualización de código en caliente (*Hot Code Upgrade*), que permite modificar una aplicación en funcionamiento sin detener el servicio, la inmutabilidad de las variables, que impide cambiar un valor una vez asignado, y la distribución transparente, que facilita la ejecución y comunicación de diferentes partes de un programa en múltiples máquinas físicas como si formaran un único sistema.

SINTAXIS

La sintaxis de Erlang está fuertemente influenciada por Prolog y se diferencia de lenguajes como C o Java porque no se basa en el anidamiento de paréntesis, sino en una estructura de puntuación que organiza el código de forma secuencial. En este lenguaje, las variables comienzan con letra mayúscula, mientras que los átomos, que representan constantes con nombre, se escriben en minúscula. Además, las listas se delimitan con corchetes (`[]`) y las tuplas con llaves (`{}`). Las funciones finalizan con un punto (`.`), las expresiones dentro de un mismo bloque se separan mediante comas (`,`), y las distintas cláusulas de una función o de estructuras como `case` e `if` se distinguen con punto y coma (`;`). Otra característica importante es el uso del *pattern matching*, donde el operador `=` no solo permite vincular valores, sino también comparar y extraer información de las estructuras de datos.

En la resolución del programa del semáforo, esta forma de organizar la sintaxis permitió definir de manera clara las transiciones entre estados, ya que cada alternativa del `case` aparece separada por punto y coma y puede interpretarse como una regla independiente. Comparado con Lisp, donde la lógica del programa depende de múltiples niveles de paréntesis anidados, el código en Erlang resultó más sencillo de leer y seguir visualmente, porque no fue necesario controlar continuamente la apertura y cierre de paréntesis, sino identificar las cláusulas delimitadas por estos símbolos, facilitando la comprensión del flujo del programa y de las distintas transiciones del semáforo.

ASPECTO SINTÁCTICO	LISP	ERLANG
FORMA BÁSICA	Listas entre paréntesis	Expresiones similares a otros lenguajes
USO DE PARÉNTESIS	Muy intensivo	Moderado
TERMINACIÓN DE SENTENCIAS	No requiere	Punto (<code>.</code>) obligatorio
DEFINICIÓN DE FUNCIONES	Mediante listas	Mediante cláusulas
LECTURA INICIAL	Puede resultar difícil	Más familiar para programadores de otros lenguajes

SEMÁNTICA

La semántica de Erlang está arraigada en los principios de la programación funcional y se caracteriza por la asignación única, la evaluación estricta y el modelo de actores. La asignación única implica que una variable es inicialmente un contenedor vacío y, una vez vinculada a un valor, este no puede modificarse, por lo que las estructuras de datos son inmutables. La evaluación estricta establece que los argumentos de una función se evalúan antes de ser pasados a ella, mientras que el modelo de actores basa la ejecución en el envío y recepción asíncrona de mensajes entre procesos independientes.

Estas características influyeron directamente en la resolución del programa del semáforo. Debido a que no es posible utilizar variables globales o estados mutables, el ciclo se simuló transmitiendo el estado a través de los argumentos de las funciones. En particular, la función `timer` no actualiza ni reemplaza el color actual, sino que lo calcula

a partir de un dato externo, el Timestamp Unix(epoch) recibido como parámetro. Mediante el operador módulo (rem) aplicado a la duración total del ciclo (216 segundos), cada nueva llamada obtiene el estado correspondiente según el instante de tiempo, permitiendo representar el avance del semáforo de manera puramente funcional, sin reasignar ni modificar ninguna variable en memoria.

Con el fin de identificar las principales diferencias y similitudes entre Erlang y Lisp, se presenta a continuación un cuadro comparativo que analiza diversos aspectos relevantes de ambos lenguajes. Entre ellos se incluyen su paradigma principal, propósito original, sistema de tipos, modelo de compilación, gestión de memoria y características relacionadas con la concurrencia y los sistemas distribuidos. Se comparan elementos propios de la programación funcional, como la evaluación, la inmutabilidad, el manejo de variables, la recursión y la manipulación de listas, así como sus aplicaciones en áreas específicas, tales como la inteligencia artificial y el desarrollo de servidores de alta disponibilidad. Este análisis permite comprender las fortalezas de cada lenguaje y las razones por las cuales fueron diseñados para resolver diferentes tipos de problemas. Además, resulta importante analizar las principales ventajas y desventajas de Erlang, ya que estas permiten comprender mejor los contextos en los que este lenguaje ofrece un mayor rendimiento y aquellos en los que puede presentar ciertas limitaciones

VENTAJAS Y DESVENTAJAS	
LISP	ERLANG
Excelente para la IA y programación	Excelente para concurrencia masiva
Sintaxis extremadamente flexible	Alta tolerancia a fallos
Permite crear DSLs fácilmente	Ideal para sistemas distribuídos
Curva de aprendizaje por la sintaxis	Menos flexible para metaprogramac
Menos utilizado en la industria actual	Menor demanda local que otros lenguajes

ASPECTO	LISP	ERLANG
PARADIGMA PRINCIPAL	Funcional, multiparadigma	Funcional concurrente
PROPÓSITO ORIGINAL	Investigación en IA	Sistemas de telecomunicaciones y distribuido
TIPADO	Generalmente dinámico	Dinámico
COMPILACIÓN	Interpretado o compilado según la implementación	Compilado a bytecode para la máquina virtual BEAM
GESTIÓN DE MEMORIA	Garbage Collector	Garbage Collector
CONCURRENCIA	Limitada según implementación	Nativa y masiva
TOLERANCIA A FALLOS	Depende de la implementación	Característica fundamental
ESCALABILIDAD	Buena	Muy alta
DISTRIBUCIÓN EN RED	No es una característica central	Diseñado para sistemas distribuidos
MANIPULACIÓN DE LISTAS	Excelente (Es una de sus bases)	Disponible pero no es el foco principal
METAPROGRAMACIÓN	Muy potente mediante macros	Limitada comparada con lisp
SISTEMAS DISTRIBUIDOS	Posible mediante bibliotecas	Integrado en el lenguaje
PROCESAMIENTO CONCURRENT	Depende de la implementación	Nativo mediante procesos ligeros
DESARROLLO DE IA	Tradicionalmente muy utilizado	Poco frecuente
SERVIDORES DE ALTA DISPONIBILIDAD	Posible	Uno de sus principales usos
EVALUACIÓN	Basada en listas y expresiones simbólicas	Basada en expresiones funcionales
INMUTABILIDAD	Recomendada pero no obligatoria	Predeterminada
VARIABLES	Pueden modificarse según la implementación	Una vez asignadas no cambian
RECURSIÓN	Muy utilizada	Fundamental
PASO DE MENSAJES	No es central	Mecanismo principal de comunicación
ESTADO COMPARTIDO	Puede existir	Evitado por diseño

CONCLUSIÓN

Como conclusión, podemos decir que el objetivo de aplicar el paradigma funcional fue cumplido. Fue un proceso de aprendizaje constante, tuvimos que aprender a manejar estados sin modificar variables, lo cual cambió nuestra forma de encarar problemas. La comparación entre Lisp y Erlang nos ayudó a entender que no existe una única forma de resolver las cosas, sino que cada lenguaje tiene su lógica propia. Investigar Erlang nos sirvió para ver cómo estos conceptos se aplican en sistemas reales y masivos, comparando su rigidez con la flexibilidad de Lisp. En conjunto, esta experiencia ha fortalecido nuestro pensamiento algorítmico, obligándonos a modelar problemas del mundo real bajo esquemas deterministas y modulares, y consolidando nuestra capacidad de investigar y adaptar herramientas de ecosistemas diversos de manera autónoma.

Bibliografía

- Armstrong, J. (2013). *Programming Erlang: Software for a Concurrent World* (2nd ed.). Pragmatic Bookshelf.
- Armstrong, J. (2010). "Erlang". *Communications of the ACM*, 53(9), 68–75.
- Cesarini, F., & Thompson, S. (2009). *Erlang Programming*. O'Reilly Media.
- [Erlang/OTP Official Documentation](#)